

Лекція 1. Вступ до DevOps

У сучасному світі інформаційних технологій, де швидкість впровадження інновацій є критичною для конкурентоспроможності, виникла необхідність у нових підходах до розробки та підтримки програмного забезпечення. Постійний розвиток цифрових технологій вимагає швидкої адаптації до змін на ринку, забезпечення стабільності IT-систем і безперервного вдосконалення програмних рішень. Традиційні моделі розробки та експлуатації, які працюють окремо одна від одної, часто стають причиною затримок, технічних помилок і непорозумінь між командами.

Development & Operations

DevOps — це *набір практик*, який поєднує розробку програмного забезпечення та IT-операцій. Він спрямований на *скорочення життєвого циклу розробки системи* та забезпечення безперервної доставки з високою якістю програмного забезпечення.



Для подолання цих викликів була розроблена методологія DevOps, яка інтегрує процеси розробки програмного забезпечення із процесами його підтримки та експлуатації. DevOps не лише прискорює впровадження нових функцій, але й дозволяє підвищити стабільність систем, знизити ризики помилок і забезпечити якісну взаємодію між усіма учасниками процесу розробки.

Цей підхід має на меті не лише технічну оптимізацію процесів, але й зміну культурного середовища в організаціях. DevOps стає ключовим елементом трансформації IT-індустрії, сприяючи швидшому випуску оновлень,

ефективнішому використанню ресурсів і, в кінцевому підсумку, підвищенню задоволеності кінцевих користувачів.

DevOps – це сучасна методологія, яка виникла як відповідь на зростаючі виклики в галузі інформаційних технологій, де важливими факторами є швидкість, якість і гнучкість. Назва «DevOps» походить від поєднання слів *Development* (розробка) та *Operations* (експлуатація), що символізує інтеграцію цих двох важливих складових життєвого циклу програмного забезпечення.

DevOps не є лише набором технологій або інструментів – це передусім культурна трансформація в організаціях, яка змінює підходи до взаємодії між командами розробників, тестувальників, операційних інженерів та інших зацікавлених сторін. Традиційно розробка програмного забезпечення та його експлуатація виконувались різними командами, що часто призводило до конфліктів, затримок і помилок через різні пріоритети, інструменти та методи роботи. DevOps спрямований на усунення цих бар'єрів шляхом створення спільної культури, де всі працюють над однією метою – швидкою та стабільною доставкою програмного забезпечення, що відповідає вимогам користувачів.

Основною метою DevOps є автоматизація, стандартизація та оптимізація всіх етапів життєвого циклу розробки програмного забезпечення (ПЗ):

- *створення коду*: забезпечення прозорості та інтеграції коду через системи управління версіями, такі як Git;
- *тестування*: впровадження автоматизованого тестування для виявлення помилок на ранніх етапах розробки;
- *збірка та розгортання*: автоматизація процесу інтеграції (Continuous Integration) та доставки оновлень (Continuous Delivery);
- *експлуатація*: підтримка високої доступності та стабільності систем за допомогою моніторингу та автоматичного масштабування.

DevOps також ґрунтується на принципі безперервного вдосконалення, який передбачає регулярний аналіз процесів, отримання зворотного зв'язку від користувачів та адаптацію до нових викликів. Завдяки цим підходам DevOps

дозволяє скоротити час між створенням нової функції або виправленням помилки та її впровадженням у робоче середовище.

Крім того, DevOps інтегрує використання спеціалізованих інструментів, таких як Docker, Jenkins, Kubernetes, які сприяють автоматизації й стандартизації процесів. Це робить його одним із найпотужніших підходів до управління програмними проектами в сучасній ІТ-галузі.

Таким чином, DevOps можна визначити як симбіоз культурних змін, технічних інновацій та інструментів, які разом забезпечують ефективну розробку та експлуатацію програмного забезпечення.

Процес DevOps охоплює низку етапів, які забезпечують безперервну інтеграцію, доставку та моніторинг програмного забезпечення. Кожен етап має на меті оптимізацію співпраці між командами розробки та операцій, забезпечуючи швидке, якісне та безпечне впровадження змін.

Основні етапи процесу DevOps

Етап	Опис
Планування	Визначення вимог до продукту, планування функцій і завдань для реалізації
Розробка	Написання коду, створення нових функцій та інтерфейсів
Тестування	Виконання автоматизованих тестів для перевірки якості та стабільності продукту
Розгортання	Автоматичне розгортання коду в продуктивне середовище після успішного тестування
Моніторинг	Безперервний моніторинг продуктивності та стабільності системи після розгортання
Зворотний зв'язок	Збір даних від користувачів та адаптація продукту згідно з отриманими результатами

DevOps ґрунтується на впровадженні чотирьох ключових процесів, які забезпечують безперервний розвиток, тестування, доставку та моніторинг програмного забезпечення. Ці процеси сприяють швидкості, якості та стабільності розробки, створюючи основу для ефективної роботи DevOps-команди.

Безперервна інтеграція (Continuous Integration, CI) є важливою частиною DevOps-практики, яка забезпечує систематичний підхід до інтеграції змін у

базовий код додатку. Вона спрямована на підтримання стабільності проєкту, мінімізацію ризиків помилок і забезпечення постійної готовності до подальших етапів розробки та доставки програмного забезпечення.

Основні принципи безперервної інтеграції:

- *часте об'єднання змін*: розробники інтегрують свої зміни до центрального репозиторію кілька разів на день. Це дозволяє уникнути проблем, пов'язаних із великими, рідкими оновленнями коду, які можуть спричиняти конфлікти;
- *автоматичне тестування*: з кожним комітом система автоматично запускає набір тестів (юніт-, інтеграційних тощо). Це допомагає виявляти помилки на ранньому етапі;
- *системи контролю версій*: використання таких інструментів, як Git, забезпечує організоване управління змінами, дозволяє відстежувати історію змін і легко повертатися до попередніх версій коду, якщо виникають проблеми;
- *автоматизація процесу збірки*: інструменти CI забезпечують автоматичну збірку програми після кожного коміту, що дозволяє виявляти помилки ще до етапу тестування.

Безперервна доставка (Continuous Delivery, CD) є важливим етапом у DevOps-циклі, який фокусується на автоматизації всіх процесів, пов'язаних з доставкою та впровадженням змін у робоче середовище. CD забезпечує надійне і швидке розгортання готового коду в тестових або виробничих середовищах, знижуючи ризики і підвищуючи ефективність усієї команди.

Основні принципи безперервної доставки:

- *готовність коду до розгортання в будь-який момент*: після того як код пройшов автоматизоване тестування в рамках Continuous Integration, він автоматично стає готовим до розгортання. Кожна зміна в коді, яка проходить через усі етапи тестування, є готовою для потенційного випуску без додаткових перевірок;

- *автоматизація процесу розгортання*: після того, як код успішно пройшов тестування, він автоматично розгортається у відповідному середовищі – від тестового до передвиробничого. Процес розгортання автоматизується, що мінімізує людський фактор і забезпечує швидке і безпомилкове розгортання на різних етапах. Проте останній етап (впровадження в виробниче середовище) може бути виконаний вручну, якщо є потреба в додатковій перевірці або контролі;
- *відокремленість процесів розгортання та впровадження*: хоча автоматизація дозволяє зменшити кількість помилок, рішення про остаточне впровадження в реальне середовище часто залишається за розробниками або операційною командою. Це дозволяє зберегти контроль над критичними моментами, але в той же час дозволяє максимально скоротити час на підготовку до релізу.

Безперервна доставка (CD) є важливою складовою частиною сучасних DevOps-практик, що забезпечує автоматизоване і швидке впровадження змін до програмних продуктів у різні середовища. Для досягнення цієї мети використовуються потужні інструменти, які дозволяють автоматизувати процеси упаковки, розгортання та масштабування додатків. Вони значно спрощують і прискорюють розгортання, забезпечуючи узгодженість між тестовими і виробничими середовищами, а також підвищуючи ефективність управління інфраструктурою.

Безперервне тестування (Continuous Testing, CT) є важливою складовою DevOps-процесів, що забезпечує автоматизовану перевірку якості коду на всіх етапах розробки. Цей процес включає тестування на кожному етапі CI/CD-конвеєра, починаючи від першої інтеграції змін у репозиторій до фінального розгортання в виробничому середовищі. Безперервне тестування допомагає виявляти помилки на ранніх етапах і забезпечує високий рівень якості коду.

Основні принципи безперервного тестування:

- *інтеграція тестування в кожен етап CI/CD-конвеєра*: тестування автоматично інтегрується на всіх етапах процесу розробки: від

безперервної інтеграції (CI) до безперервної доставки (CD). Після кожного коміту коду автоматично запускаються відповідні тести, що дозволяє виявляти помилки на ранніх етапах і не допустити їх на наступні етапи розробки.

- *виконання різних типів тестів*: безперервне тестування охоплює різні типи тестів;
- *автоматичний аналіз результатів тестів*: після виконання тестів результати автоматично аналізуються для виявлення потенційних проблем. Система повідомляє команду розробників про виявлені помилки або невідповідності стандартам якості, що дозволяє оперативно вирішувати проблеми і підтримувати стабільність коду.

Безперервне тестування є важливим етапом у процесах DevOps, що дозволяє забезпечити високу якість коду на всіх етапах розробки. Для автоматизованого тестування використовуються різноманітні інструменти, які допомагають ефективно перевіряти функціональність програмного забезпечення і виявляти помилки на ранніх етапах. Вони забезпечують виконання різних типів тестів, від юніт-тестів до складніших інтеграційних тестів, тим самим підтримуючи якість коду та дозволяючи швидко реагувати на зміни.

Безперервний моніторинг (Continuous Monitoring, CM) є важливою складовою DevOps-процесів, що дозволяє відстежувати стан додатків, інфраструктури та всієї системи в реальному часі. Це забезпечує стабільну роботу програмного забезпечення та дозволяє швидко виявляти й усувати проблеми, що виникають в експлуатаційних середовищах. Безперервний моніторинг не лише підвищує стабільність системи, а й допомагає виявляти ефективність роботи додатків та інфраструктури, зменшуючи час простою і покращуючи досвід користувачів.

Основні принципи безперервного моніторингу:

- *моніторинг продуктивності, навантаження, журналів помилок і доступності системи*: безперервний моніторинг включає відстеження ключових показників продуктивності (наприклад, час відгуку,

використання ресурсів, пропускну здатність) і навантаження на систему. Він також включає моніторинг журналів помилок та доступності, що дозволяє швидко виявляти неполадки та реагувати на них до того, як вони вплинуть на кінцевого користувача;

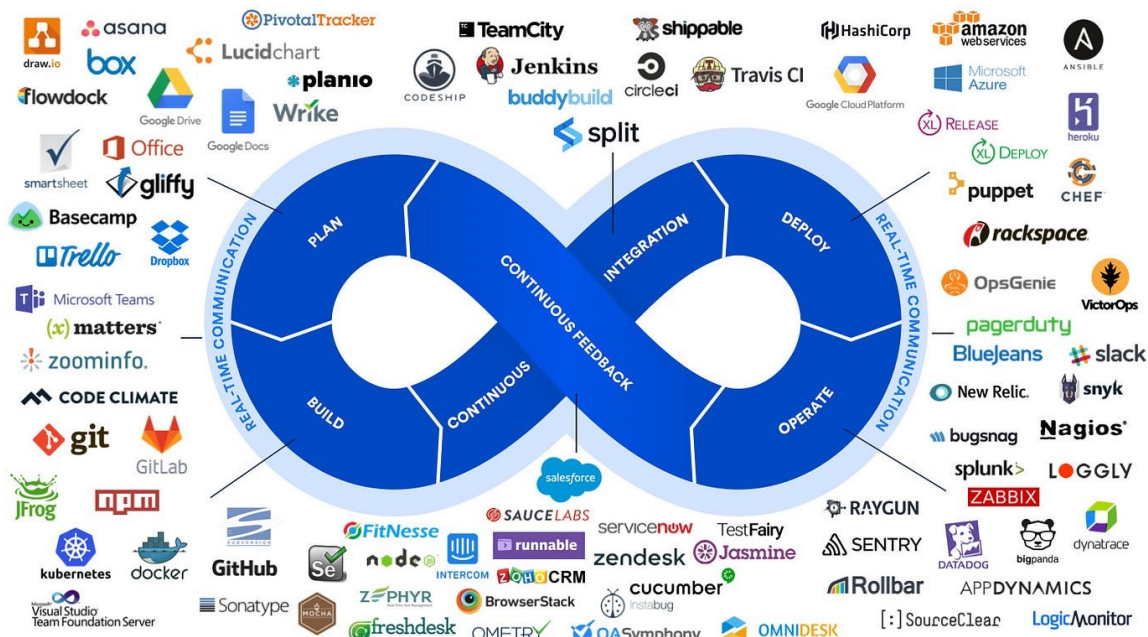
- *використання алертів для оперативного реагування на збої*: для оперативного реагування на збої та проблеми в системі використовуються алерти, які спрацьовують при досягненні певних порогових значень. Наприклад, якщо система досягає максимального порогу навантаження або виникає помилка на рівні інфраструктури, моніторингова система автоматично сповіщає відповідальних осіб або автоматично ініціює певні коригувальні дії;
- *збір даних для покращення майбутніх релізів*: безперервний моніторинг забезпечує збір та аналіз даних, що дає можливість виявляти вузькі місця в системі, а також прогнозувати потенційні проблеми у майбутньому. Ці дані використовуються для вдосконалення процесу розробки та оптимізації продуктів у наступних релізах.

Порівняння основних переваг DevOps

Перевага	Опис	Вплив на бізнес
Швидкість	Прискорення циклу розробки через автоматизацію та безперервну інтеграцію	Швидший вихід продуктів на ринок, перевага перед конкурентами
Якість	Автоматизоване тестування і безперервне вдосконалення дозволяють знижувати кількість помилок	Покращення стабільності та зниження витрат на виправлення помилок
Гнучкість	Можливість швидко адаптуватися до змін та масштабувати системи	Підвищення конкурентоспроможності та ефективності
Ефективність	Автоматизація та оптимізація використання ресурсів	Зменшення витрат, підвищення ефективності
Інновації	Швидке створення та тестування нових ідей	Покращення досвіду користувачів, розвиток інновацій
Поліпшення командної роботи	Спільні інструменти та процеси покращують співпрацю між командами	Зростання задоволеності працівників і ефективності роботи

Інструменти DevOps

Для ефективного впровадження принципів DevOps використовується широкий набір інструментів, кожен із яких оптимізує певний етап життєвого циклу програмного забезпечення. Ці інструменти автоматизують рутинні процеси, підвищують прозорість і забезпечують стабільність систем.



Для управління конфігураціями

Управління конфігураціями – це ключовий аспект DevOps, який дозволяє забезпечити стандартизоване та автоматизоване налаштування інфраструктури.

Ansible: легкий у використанні інструмент, що дозволяє автоматизувати розгортання, управління конфігураціями та оркестрацію. Його простий синтаксис YAML спрощує створення скриптів навіть для новачків;

Puppet: інструмент для автоматизації управління інфраструктурою, який дозволяє визначати конфігурації через декларативну мову. Він забезпечує контроль над великими системами завдяки централізованому управлінню;

Chef: потужний інструмент, який використовує мову програмування Ruby для опису конфігурацій, забезпечуючи гнучкість у налаштуванні складних систем.

Для автоматизації CI/CD

Безперервна інтеграція (CI) і безперервна доставка (CD) є основними компонентами DevOps, які забезпечують швидке тестування та впровадження змін.

Jenkins: популярна платформа з відкритим кодом, яка підтримує безліч плагінів для автоматизації процесів CI/CD. Її гнучкість дозволяє налаштувати складні конвеєри доставки програмного забезпечення;

GitLab CI/CD: інтегрована платформа, яка забезпечує повний цикл розробки – від управління кодом до автоматизації тестування та доставки;

CircleCI: Хмарний сервіс для автоматизації CI/CD, який підтримує гнучкі сценарії розгортання та інтегрується з популярними системами контролю версій, такими як GitHub.

Для контейнеризації

Контейнеризація стала невід'ємною частиною DevOps, оскільки вона дозволяє ізолювати додатки та їхні залежності, забезпечуючи портативність і легкість у масштабуванні.

Docker: інструмент для створення контейнерів, який дозволяє упакувати додаток із усіма необхідними залежностями в стандартизоване середовище, незалежне від платформи;

Kubernetes: система оркестрації контейнерів, яка дозволяє автоматизувати розгортання, масштабування та управління контейнеризованими додатками в кластерному середовищі. Kubernetes забезпечує балансування навантаження та самовідновлення систем.

Для моніторингу

Моніторинг є критичним для забезпечення стабільності та продуктивності програмного забезпечення. DevOps використовує інструменти, які дозволяють відстежувати стан системи в реальному часі та оперативно реагувати на збої.

Prometheus: інструмент для збору метрик і моніторингу системи, який підтримує створення алертів і інтеграцію з іншими інструментами, такими як Grafana;

Grafana: графічна платформа для візуалізації даних і створення інтерактивних дашбордів. Використовується для аналізу продуктивності системи;

ELK Stack (Elasticsearch, Logstash, Kibana): комплексний набір інструментів для збору, аналізу та візуалізації логів, що дозволяє ефективно діагностувати проблеми та відстежувати аномалії.

Для управління версіями

Управління версіями коду є важливим компонентом DevOps для забезпечення прозорості, колективної роботи та історії змін.

Git: найпопулярніша система контролю версій, яка забезпечує зручну співпрацю між командами через децентралізовану модель;

GitHub/GitLab/Bitbucket: платформи, які дозволяють зберігати репозиторії, керувати кодом, інтегрувати CI/CD та організовувати спільну роботу над проектами.

Для тестування

Автоматизоване тестування є ключовим для забезпечення якості коду в DevOps.

Selenium: інструмент для автоматизованого тестування веб-додатків, що підтримує різні браузерери та мови програмування;

JUnit: бібліотека для модульного тестування Java-додатків, яка дозволяє створювати та виконувати тести;

TestNG: потужний інструмент для автоматизації тестування з підтримкою паралельного виконання тестів і інтеграції з іншими DevOps-інструментами.

Для управління інфраструктурою як кодом (IaC)

Інфраструктура як код дозволяє створювати, конфігурувати та управляти інфраструктурою через програмні скрипти.

Terraform: інструмент для опису інфраструктури як коду, який підтримує різні платформи (AWS, Azure, Google Cloud);

CloudFormation: інструмент від AWS, що дозволяє управляти ресурсами хмарної інфраструктури через декларативні шаблони.

Інструменти DevOps працюють у тісній взаємодії, забезпечуючи автоматизацію, інтеграцію та моніторинг на всіх етапах життєвого циклу програмного забезпечення. Вибір інструментів залежить від потреб організації, масштабу проєкту та технічної інфраструктури, але їхнє впровадження забезпечує підвищення продуктивності, якості та швидкості розробки.

DevOps як концепція поведінки, розширення Agile Methodology

Agile – це сучасний підхід до розробки програмного забезпечення, який спрямований на досягнення гнучкості, ефективної співпраці, швидкої адаптації до змін і постійного вдосконалення продукту. Він з'явився як реакція на недоліки традиційних методів розробки, зокрема «водоспадної» моделі (Waterfall), яка передбачала жорстку послідовність етапів проєкту – від визначення вимог до тестування й розгортання. Основні виклики водоспадного підходу, такі як повільна реакція на змінювані потреби, ризикованість фінальної стадії проєкту та відсутність тісної взаємодії між замовниками і командою, створили потребу у впровадженні більш динамічних і адаптивних рішень.

Agile орієнтований на часті ітерації, під час яких команда розробників створює й вдосконалює робочий продукт. Його основою став Маніфест гнучкої розробки програмного забезпечення (Agile Manifesto), який було опубліковано в 2001 році групою провідних інженерів і консультантів. У цьому документі визначено ключові цінності та принципи Agile, які формують його ідеологію.

Agile став відповіддю на зростаючу складність розробки програмного забезпечення в умовах динамічного розвитку технологій та бізнесу. Традиційні методи розробки часто виявлялися неефективними в ситуаціях, коли вимоги замовника змінювалися в процесі реалізації проєкту. Крім того, такі підходи створювали значні ризики:

- Тривалий час до випуску першої версії продукту;
- Висока вартість виправлення помилок, виявлених на пізніх етапах;
- Відсутність можливості отримати швидкий зворотний зв'язок від клієнтів або користувачів.

Agile вирішує ці проблеми, роблячи акцент на гнучкості процесів, взаємодії між командою та замовником і постійній доставці цінності для кінцевого користувача.

У Маніфесті Agile визначено чотири ключові цінності, які відрізняють цей підхід від традиційних моделей:

- *Люди та взаємодія важливіші за процеси та інструменти.* Головна увага приділяється командній роботі, комунікації та взаємодії, які сприяють досягненню поставлених цілей.
- *Робоче програмне забезпечення важливіше за детальну документацію.* Пріоритетом є створення продукту, який працює і відповідає реальним потребам клієнта, а не складання детальних описів або інструкцій.
- *Співпраця з клієнтами важливіша за контрактні переговори.* Agile передбачає тісну взаємодію з клієнтами на всіх етапах проєкту, що дозволяє забезпечити їхню залученість і задоволення результатом.
- *Готовність до змін важливіша за дотримання початкового плану.* Команди повинні бути здатними швидко адаптуватися до нових вимог або умов ринку, що гарантує актуальність продукту.

Окрім ключових цінностей, Agile базується на 12 принципах, які конкретизують підхід до організації роботи, управління проєктами, адаптації до змін і забезпечення якості продукту. Ці принципи є основою для впровадження Agile і допомагають командам ефективно працювати навіть у складних умовах. Нижче наведено розгорнутий опис кожного з принципів:

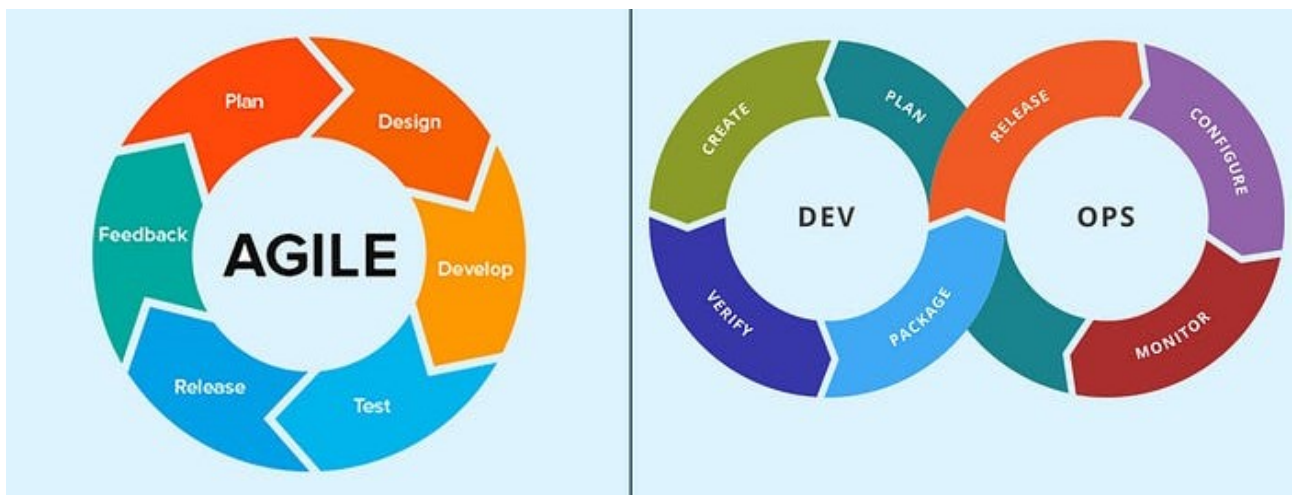
- *Задоволення клієнта через швидку та безперервну доставку цінного програмного забезпечення.* Основна мета Agile – зробити клієнта задоволеним за рахунок швидкого надання продукту, який відповідає його вимогам. Команди забезпечують регулярне оновлення програмного забезпечення, щоб максимально швидко реалізувати необхідні функціональні можливості;
- *Готовність до змін навіть на пізніх етапах розробки.* Agile підтримує гнучкість і здатність реагувати на зміну вимог у будь-який момент.

Зміни сприймаються не як загроза, а як можливість покращити продукт, зробивши його актуальним і корисним;

- *Часте постачання робочого продукту.* Продукт доставляється невеликими, але робочими частинами через короткі проміжки часу (ітерації або спринти). Це дозволяє швидко отримувати зворотний зв'язок і вчасно впроваджувати коригування;
- *Тісна співпраця між замовником і командою.* Agile передбачає постійну взаємодію із замовником для уточнення вимог, контролю прогресу та врахування змін. Це забезпечує високий рівень прозорості та довіри;
- *Мотивація і підтримка команди.* Успіх проєкту залежить від мотивованої, кваліфікованої та автономної команди. Agile сприяє створенню умов, у яких члени команди можуть самостійно приймати рішення, зберігаючи мотивацію та відчуття відповідальності;
- *Фокус на живій комунікації.* Основним засобом передачі інформації є особисте спілкування (очне або віртуальне). Це дозволяє швидко вирішувати проблеми, уникати непорозумінь і забезпечувати ефективну координацію між усіма учасниками;
- *Робочий продукт як основний показник прогресу.* В Agile акцент робиться на наявність робочого програмного забезпечення як головного критерію успішності, а не на документації чи звітах;
- *Стійкий темп розробки.* Agile підтримує стабільний, збалансований робочий ритм, який допомагає уникати вигорання команди та забезпечує довгострокову ефективність;
- *Увага до технічної досконалості та якісного дизайну.* Постійний фокус на технічному вдосконаленні та підтримці чистоти коду дозволяє створювати програмне забезпечення, яке легко підтримувати, масштабувати й розвивати.
- *Простота.* Agile орієнтується на максимальну ефективність, уникаючи зайвих дій і витрат. Принцип «зробити тільки необхідне» дозволяє зосередитися на найважливіших аспектах проєкту;

- *Самоорганізовані команди.* Команди, які працюють за Agile, самостійно визначають, як найкраще досягти поставлених цілей. Це сприяє інноваціям, високій залученості та відповідальності;
- *Постійний аналіз і вдосконалення.* Регулярні ретроспективи та аналіз процесів дозволяють команді визначати слабкі місця, запроваджувати покращення й досягати вищих результатів.

Ці 12 принципів створюють чіткі орієнтири для роботи команд, допомагають будувати довірчі відносини із замовниками та забезпечують адаптивність до мінливих умов. Вони є основою для популярних фреймворків, таких як Scrum, Kanban, SAFe та інших, і дозволяють командам досягати високих результатів навіть у складних і швидкозмінних проєктах.



Обидва цикли ілюструють безперервний процес, що сприяє ефективній розробці та підтримці програмного забезпечення.

DevOps можна розглядати як логічне продовження та розширення методології Agile, яке відповідає на сучасні виклики, пов'язані з масштабуванням, автоматизацією та безперервним вдосконаленням систем. Обидві концепції мають на меті оптимізацію процесів розробки та доставки програмного забезпечення, забезпечуючи швидку реакцію на зміни й орієнтацію на потреби клієнтів.

Agile зосереджений на організації командної роботи, створенні гнучких ітеративних процесів розробки та швидкому наданні функціонального продукту.

Ця методологія пропонує інструменти для поліпшення взаємодії між розробниками, замовниками й іншими зацікавленими сторонами. Вона орієнтована на отримання швидкого зворотного зв'язку, що дає змогу оперативно коригувати продукт у відповідь на зміни у вимогах чи умовах ринку.

DevOps додає до цих принципів новий вимір, акцентуючи увагу на інтеграції операційних процесів і створенні умов для безперервного вдосконалення системи на всіх етапах її життєвого циклу. DevOps об'єднує розробників (Dev) і фахівців з експлуатації (Ops), усуваючи бар'єри між цими командами для досягнення спільної мети – швидкого та стабільного постачання програмного забезпечення.

Якщо Agile в основному охоплює життєвий цикл розробки програмного забезпечення, то DevOps включає весь життєвий цикл продукту: від планування, розробки та тестування до розгортання й моніторингу в продуктивному середовищі. Це забезпечує повну інтеграцію всіх процесів, що дозволяє не лише створювати якісний продукт, а й гарантувати його стабільність та ефективність у роботі.

DevOps також розширює можливості Agile через:

- *Автоматизацію.* Використання інструментів для автоматичного тестування, інтеграції, розгортання та моніторингу значно пришвидшує робочі процеси, зменшує кількість помилок і забезпечує стабільність системи.
- *Моніторинг та аналітику.* Постійний моніторинг продуктивності й аналітика зібраних даних дають змогу оперативно виявляти й усувати проблеми, а також прогнозувати можливі ризики.
- *Безперервне вдосконалення.* DevOps підтримує ідею ітеративного вдосконалення, дозволяючи оновлювати та масштабувати систему з урахуванням реальних умов експлуатації.

Таким чином, DevOps не лише підтримує принципи Agile, а й розширює їхній вплив на всі аспекти розробки та експлуатації продукту. Це робить DevOps універсальним підходом для організацій, які прагнуть забезпечити високу якість,

швидкість і надійність своїх програмних рішень. У поєднанні з Agile DevOps стає потужним інструментом для забезпечення інновацій, ефективності та конкурентоспроможності на сучасному ринку.

Порівняння Agile та DevOps

Параметр	Agile	DevOps
Мета	Забезпечення гнучкої розробки та швидкого впровадження функціональних продуктів	Інтеграція процесів розробки та операцій для забезпечення безперервного циклу доставки
Сфера застосування	Зосереджено на процесах розробки програмного забезпечення	Охоплює весь життєвий цикл продукту: розробку, тестування, розгортання та підтримку
Фокус	Гнучкість у процесах розробки, швидкий зворотний зв'язок із замовником	Автоматизація, моніторинг і безперервна інтеграція для стабільної роботи продукту
Командна робота	Зосереджено на співпраці в межах команди розробників	Об'єднує команди розробки, тестування та експлуатації в єдиний цикл
Основні принципи	Ітеративний підхід, короткі спринти, постійне вдосконалення	Автоматизація, безперервна інтеграція, розгортання та моніторинг
Ключові інструменти	Jira, Trello, Asana (для планування та управління проєктами)	Jenkins, Docker, Kubernetes, Prometheus, Grafana (для автоматизації та моніторингу)
Швидкість впровадження змін	Забезпечується через ітерації та регулярний зворотний зв'язок із клієнтом	Автоматизовані процеси дозволяють швидше доставляти зміни у продуктивне середовище
Основна перевага	Швидка адаптація до змін і створення цінності для клієнта	Безперервність процесів і стабільність роботи системи
Область використання	Управління проєктами та оптимізація процесів розробки	Технічна реалізація та автоматизація всіх процесів, пов'язаних із розробкою та підтримкою